

MUST3603 Final Project Proposal

Nikolas Varga and Jake Everard

(t)riggerable (a)uditory (b)ela (ba)sed (s)ynthesis (a)nd (r)esonance (tabbasar)

Goal

For **tabbasar** we propose to create a synthesizer system including digital audio effects, physical controls, and digital timing controls. We would control aspects of the synthesizer with physical controls. These include a configurable wavetable, with detune (trimmer), octave (trimmer), level (trimmer). Additionally, we will implement a lowpass filter with adjustable frequency (trimmer), Q (trimmer), and filter type (button cycle). ADSR (trimmer x4), a gate which triggers the ADSR - (button), an arpeggiator, and a chord selector. We propose encapsulating the wavetable, filter, and ADSR in C++ classes to allow for repeatability; we further propose encapsulating the entire oscillator in a class. We will articulate a “preset” property that changes all controls to predetermined values. The arpeggiator will necessitate some type of note input– this will manifest as a physical keyboard interface with buttons, and a switch for turning on/off sequencer mode. For the filter, a stretch goal is a kind of comb filter or resonant, multipeak filter with a tunable center frequency and resonance.

The sequence of work will have both required and stretch deliverables divided by week. Deliverables will be required by the dates listed. Stretch goals can become required in the next week if it is determined to be useful in the process of building the project. To accelerate our work and make it more cohesive, the project will be stored in a GitHub repository to allow for change tracking and seamless updates between group members.

System Design

For user interface, we propose a set of 4-5 potentiometers that update the Bela GUI to control different parameters. Pressing a button will cycle the set of potentiometers through different parameter sets. For example, 1 press would allow the dials to control the ADSR of the first oscillator, then a second press would allow them to control the filter parameters. This will allow us to minimize the number of potentiometers while allowing the user to adjust many parameters of the system.

Oscillator 1 Parameters: Center frequency, detune, level, octave, waveshape (button)

Oscillator 1 ADSR: A,D,S,R

Oscillator 2 Parameters: Detune, level, semitone offset, waveshape (button)

Oscillator 2 ADSR

Additive Noise: intensity, noise enable (button)

Filter Parameters: Center frequency, Q, filter type (button)

LFO Parameters: Waveform, rate, intensity, DC offset. Mapping through GUI

Delay parameters: delay length, feedback, delay enable (button)

AM parameters: intensity, frequency, waveshape (button)

FM parameters: intensity, frequency, waveshape (button)

Arpeggiator: Step 0 semitones, step 1 st, step 2 st, step 3 st, arpeggiator enable (button)

Global controls: Arpeggiator tempo, save preset, FM B->A/independent waveshape, AM

B->A/independent waveshape

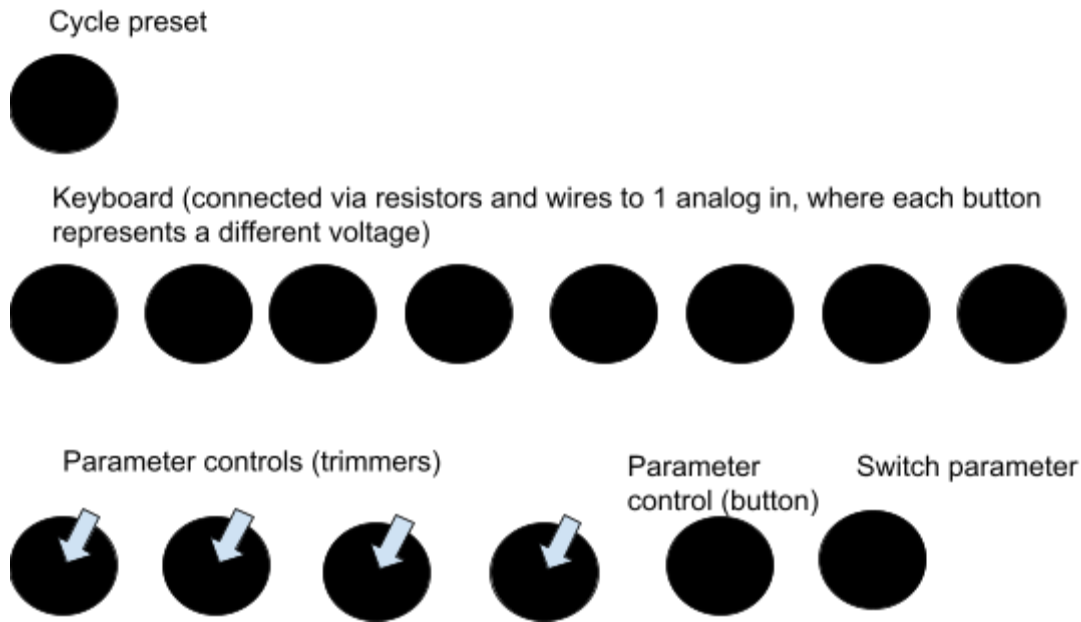


Fig. 1: Proposed user interface layout

Presentation

We will combine the sound output of tabbasar with percussion that will be arranged in Ableton. This will constitute a live performance using the system, where we will modify parameters of the system live in our performance. The system will be designed to be used in a live performance setting to demonstrate the ability to modify various parameters live.

References

From class, we will reference multiple files. For reference on building our wavetable, we will use class 6 wavetable_complete, sin_slider, class_8, and class_10 VCO. For the arpeggiator/sequencer, we will use class_10 step sequencer. For reference on envelopes, we will utilize lectures and code from class_18 ramp filters. For filters, we will look at implementation of filters from assignment 3 and use the Biquad reference on the Bela IDE sidebar. For delay, we will reference class_17 lectures and code on circular buffers. For the low-pass filter, we will be

able to reference assignment 3, but we will likely need Professor Zappi's assistance in the theoretical and practical understanding of more advanced filters like comb filters that we hope to implement. We can also reference online resources and other class resources like those on the Z-transform for these purposes.

As a general inspiration, we are referencing the VST plugin Serum by Xfer. Serum is a two oscillator wavetable synthesizer with multiple effects, modulation styles, and continuous waveform adjustment. While we may not be able to use the VST for coding reference, we will reference it in terms of user interface design and signal routing.

Schedule

Milestone 1 (11/19-11/26)

Required

- 2x discrete oscillator with discrete loadable waveshapes
 - For voices, have a vector of n oscillators that are selectively muted/unmuted
- Lowpass filter
- ADSR
 - Button trigger ADSR
- Detune for each oscillator
- Semitone offset for oscillator 2 (Serum)
- Arpeggiator and sequencer (sequencer is necessary for arpeggiator)
 - Global tempo control
 - Control over (n dials) step intervals
- Additive noise

- Button “piano keyboard” as input

Stretch

- Continuous waveshape adjustment (Serum)

Milestone 2 (11/26)

Required

- Amplitude Modulation
- Frequency Modulation
- LFO class
 - Waveform, rate, intensity
- Delay
- Preset

Stretch

- Enclosure
- Resonant comb filter